CT2003 Object-Oriented Design & Programming, 26S1 · College of Computing and Data Science, NTU

Restaurant Rush

A turn-based restaurant management game in Java

Lab group COMP1 · Group 3 · Submitted Fri 13 Nov 2026

Member	Matric no.	Area
Muhamad Rifqi bin Zulkiflie	U2620160H	Customers (Regular, VIP, Critic), satisfaction; UML class diagram
Muhammad Danial Haziq bin Burhan	U2620923J	MenuItem, Menu, ComboMeal, Order, Chef, Cashier, Money, PricingPolicy; design discussion
Muhammad Kenz s/o Jafar	U2620772C	Table, Staff, Waiter, Host, Restaurant, console display and input; testing and functional evidence
Teo Soon How	U2620320K	Turn sequence, scoring, Happy Hour timing; stage tags, README, report

Repository: https://github.com/armaldro/2003-Group3 · Tags: `stage-1`, `stage-2`, `stage-3`

1. Introduction

Restaurant Rush is a command-line game in which the player manages one Waiter, one Chef and one Cashier (and, from Stage 3, one Host) over 18 turns. Each turn the player decides whom to seat, whose order to take, what to cook and what to serve. Victory after turn 18 needs at least 4 paid customers, at least $45.00 revenue and an average satisfaction of paid customers of at least 60.

Stage	Tag	Adds
1. Basic operation	`stage-1`	Regular customers; Salad $9.00, Burger $15.00, Pasta $18.00 (2 units); Waiter, Chef, Cashier; 2 tables; the full 6-step turn; Victory and Defeat
2. Customer behaviour	`stage-2`	VIP (−5 a turn, 10% off, arrives turn 4) and Critic (−12 a turn, always orders the dish that takes longest to cook, arrives turn 10)
3. New role and menu policy	`stage-3`	Host (seats one waiting customer a turn); ComboMeal (Salad + Pasta at 90% = $24.30, 3 units); Happy Hour (once per game, 2 turns, 20% off orders taken, +15 satisfaction to waiting customers). The Critic now arrives on turn 7 and orders the combo

Design approach[1]: the code follows the Entity-Control-Boundary (ECB) pattern in five packages: `entity`, `control`, `boundary`, `enums` and `util`. Game rules live in entity and control classes, never in the console classes, and all money is held as whole cents and formatted only by `Money`.

[1] Drafted by Claude (Anthropic) from the repository (feature/stage-3 merged with dev, 13 Nov 2026) and the members' report/design-discussion files; reviewed and edited by Soon How. Prompt and reply logged in gai-paper-trail/soonhow-claude-log.md.

2. Final class diagram

Figure 1 shows the whole design on one page. Classes are grouped by package (boundary, control, entity, enums, util). Abstract classes and abstract methods are in italics, static members are underlined, and visibility is shown as + (public), − (private) and # (protected). The yellow notes A to D mark the OO concepts explained below the checklist.

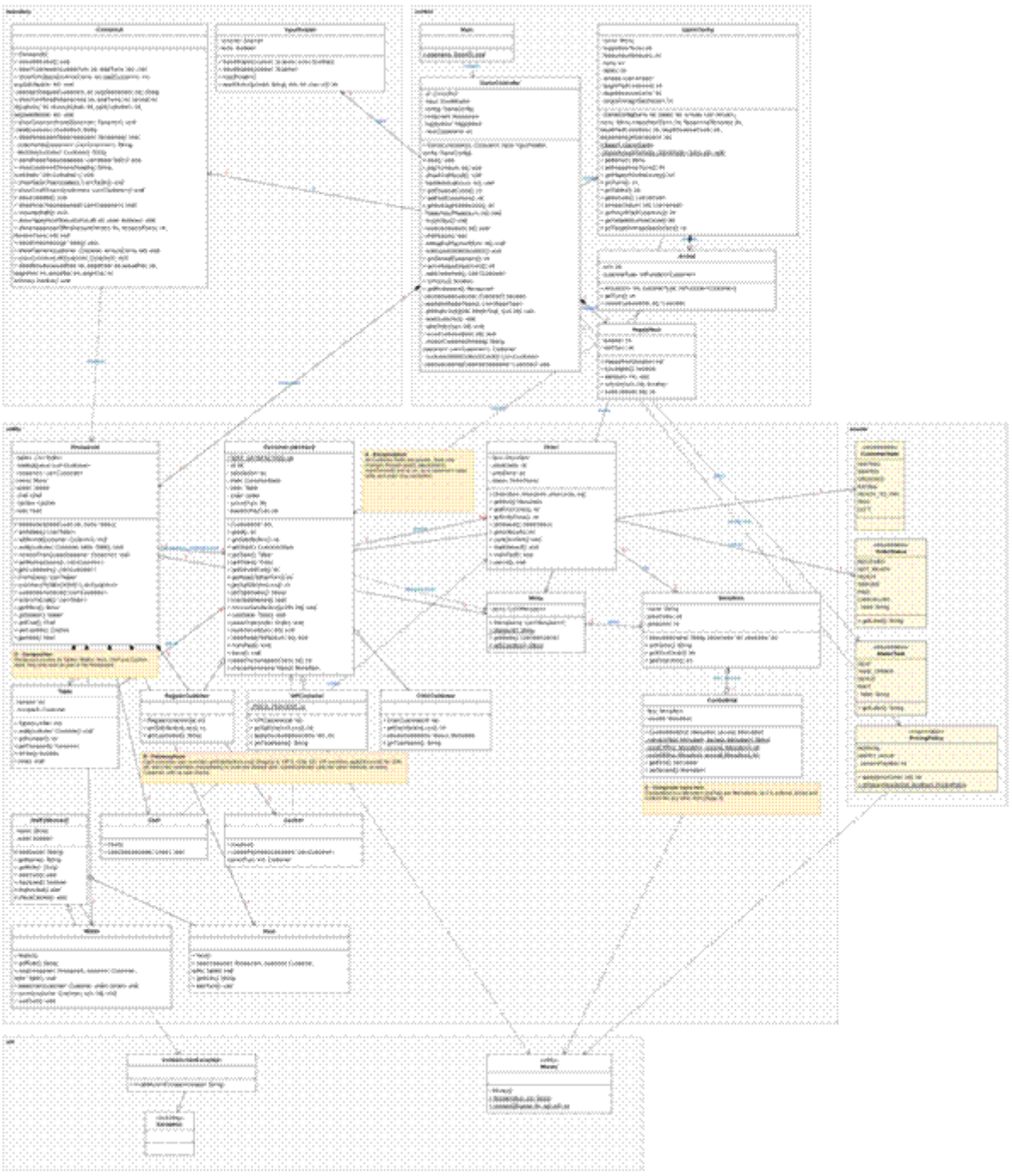


Figure 1: Final class diagram. Source file: diagrams/RestaurantRush-ClassDiagram.drawio[1]

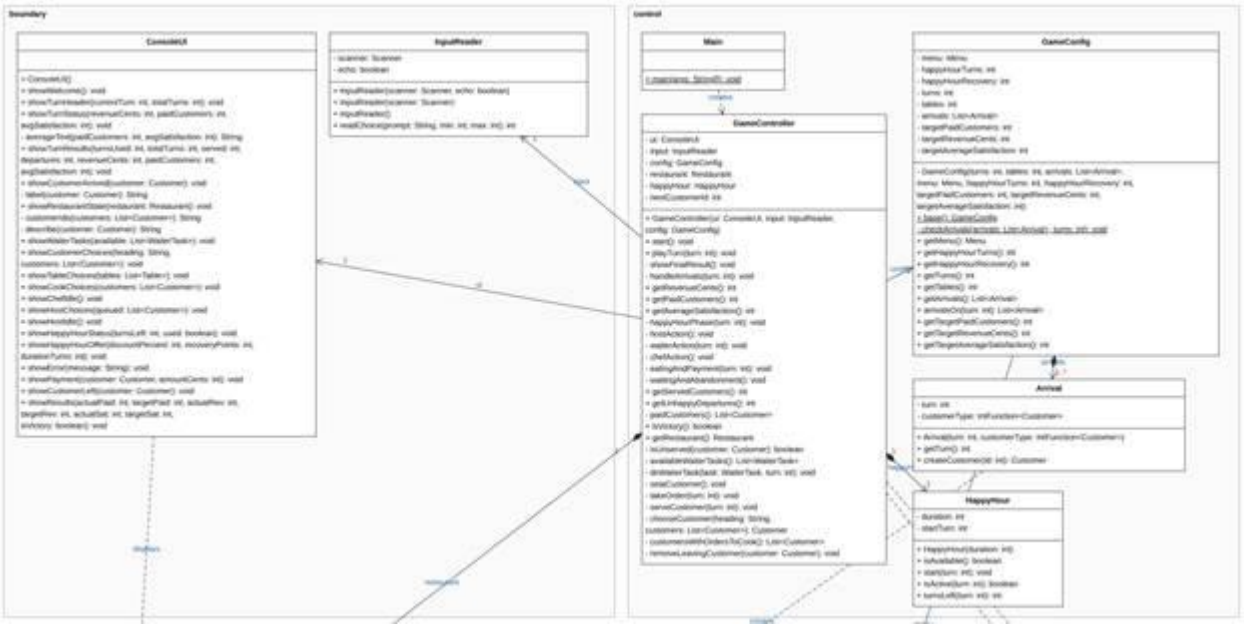


Figure 1a: boundary and control packages (ConsoleUI, InputReader, Main, GameController, GameConfig, Arrival, HappyHour)

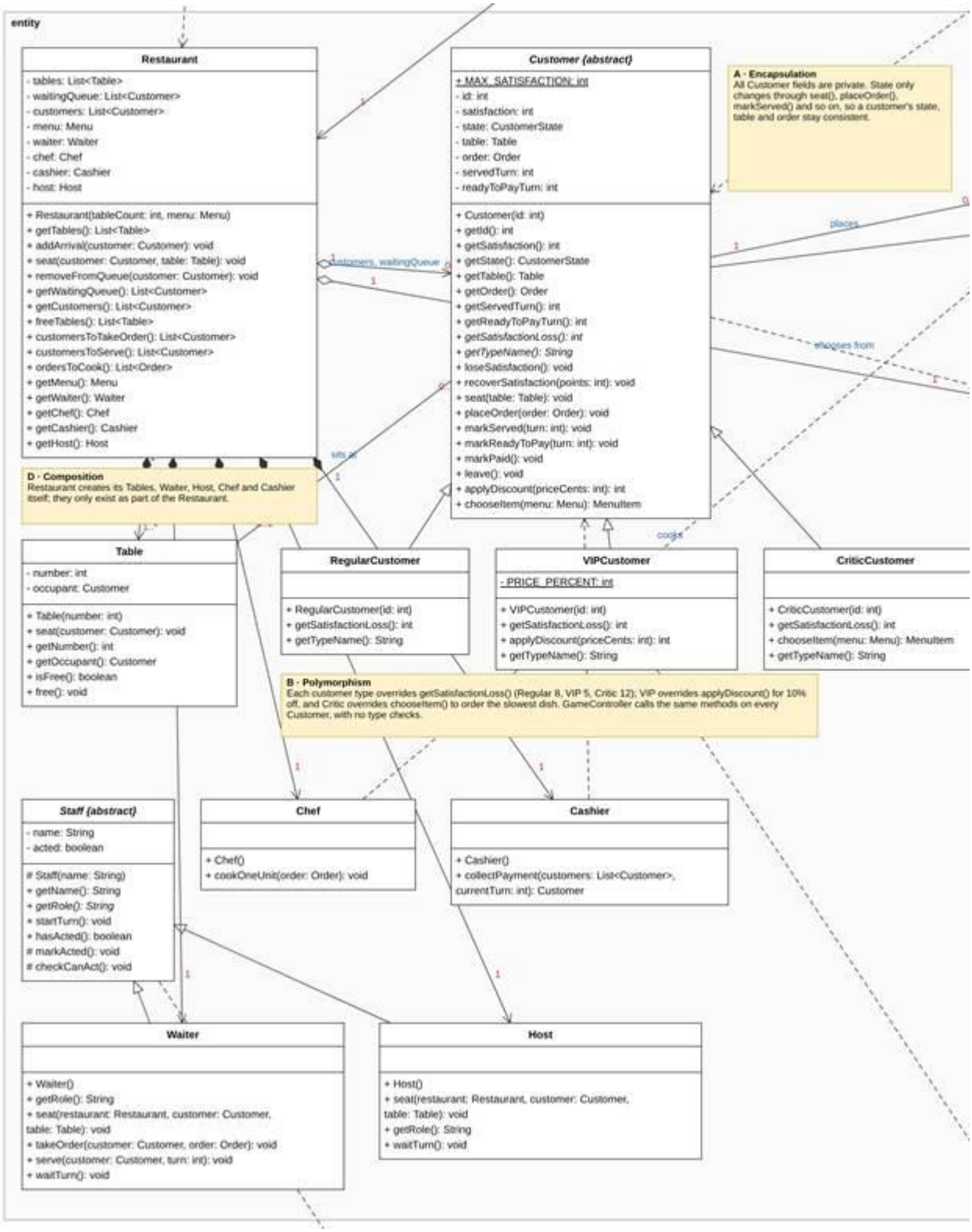


Figure 1b: entity package, part 1: Customer hierarchy, Restaurant, Table and Staff hierarchy

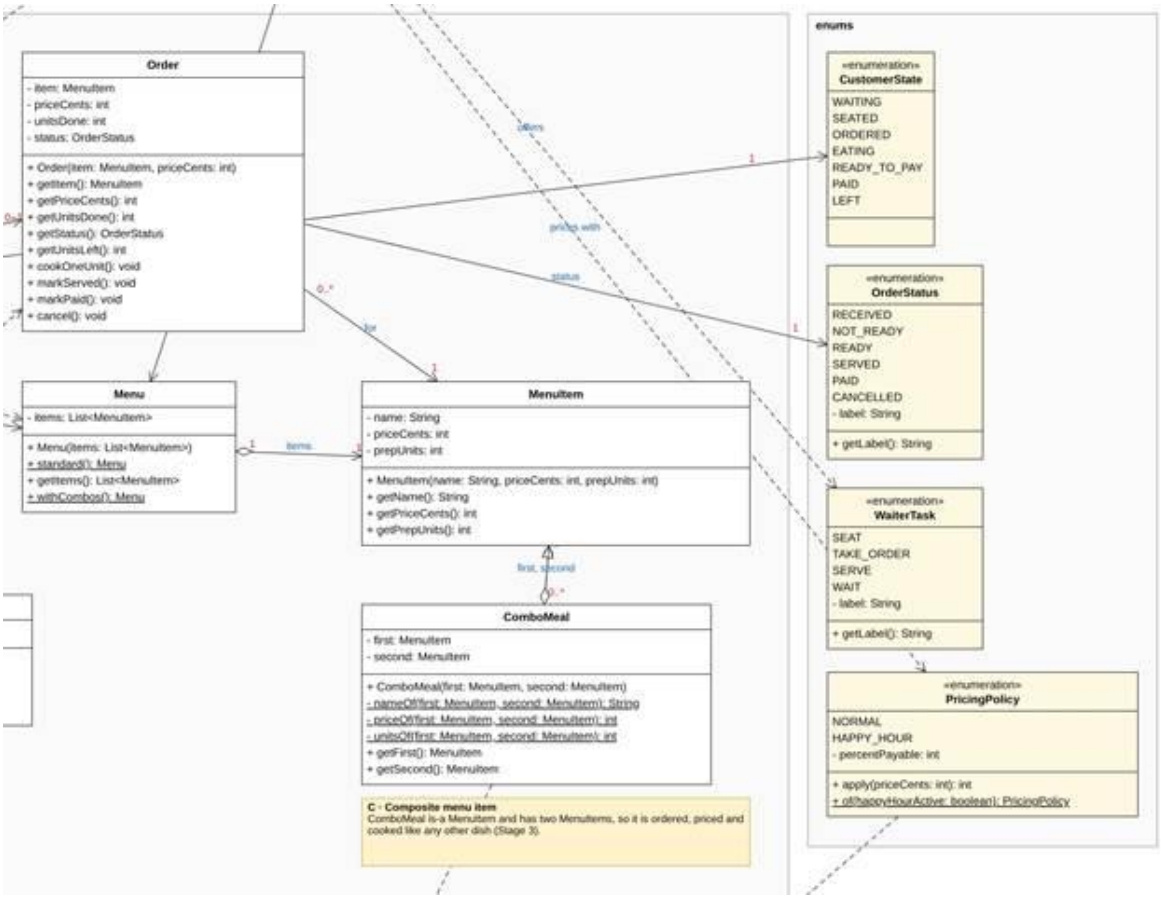


Figure 1c: entity package, part 2 (Order, Menu, MenuItem, ComboMeal) and enums



Figure 1d: util package (Money, InvalidActionException)

Checklist for the diagram

☑ Principal entity classes, plus the control and boundary classes they need

☑ Relevant enums (CustomerState, OrderStatus, WaiterTask, PricingPolicy)

☑ Key attributes and operations, with visibility (+ − #)

☑ Inheritance (Customer, MenuItem, Staff hierarchies)

☑ Associations with multiplicities where relevant (e.g. Restaurant 1 — 2 Table)

☑ No class in the diagram that is missing from the code, and no stage-3 class missing from the diagram

[1] Rifqi: the class diagram was generated by Claude (Anthropic) from the compiled code, then reviewed and laid out by me. See Section 8.1 and gai-paper-trail/rifqi-claude-log.md.

Annotated OO concepts

Each row matches the yellow note with the same letter in Figure 1.

#	Concept	Where in the diagram	What it shows
A	Encapsulation	Order; Customer	Order's price is fixed when the order is taken (set in the constructor, no setter), and its status changes only through `cookOneUnit()`, `markServed()`, `markPaid()` and `cancel()`. Customer works the same way: all fields are private and change only through `seat()`, `placeOrder()`, `markServed()`, `markReadyToPay()`, `markPaid()` and `leave()`, so a customer's state, table and order stay consistent.
B	Polymorphism	Customer ← RegularCustomer, VIPCustomer, CriticCustomer	Each subclass overrides `getSatisfactionLoss()` (Regular 8, VIP 5, Critic 12) and `getTypeName()`; VIPCustomer also overrides `applyDiscount()` (10% off) and CriticCustomer overrides `chooseItem()` (slowest dish). GameController calls these on any Customer and never checks a customer's type.
C	Inheritance with aggregation	ComboMeal —▷ MenuItem; ComboMeal ◇— 2 MenuItem	A ComboMeal is a MenuItem and holds two MenuItems; it adds their preparation units and charges 90% of their combined price, so it is ordered, cooked and paid for like any other dish. It is drawn as aggregation, not composition, because the two dishes are passed in and also exist on the menu on their own.
D	Composition	Restaurant ◆— 1..* Table, Waiter, Chef, Cashier, Host	Restaurant creates its Tables (in its constructor) and its Waiter, Chef, Cashier and Host (as field initialisers); they exist only as part of the Restaurant.

3. Design discussion

Compiled from each member's report/design-discussion/<name>.md. Class names match the class diagram and the stage-3 code.

3.1 Key abstractions and their responsibilities

Class	Pkg	Responsibility (and what it deliberately does not do)	Collaborates with	Stage	Owner
Customer (abstract)[2]	entity	Shared customer data and rules: id, satisfaction (0 to 100), state, table and order, changed only through one method per event. Applies its own waiting loss (never below 0) and Happy Hour recovery (never above 100) and picks its own dish. Does not decide when events happen, free tables, cancel orders or count revenue.	Table, Order, Menu, MenuItem, CustomerState	1 (recover in 3)	Rifqi
RegularCustomer / VIPCustomer / CriticCustomer	entity	Override only what differs. Regular: −8 a turn, full price. VIP: −5, 10% off through `Money.percentOf`. Critic: −12, full price, orders the dish with the most preparation units (Pasta in Stage 2, the Salad + Pasta combo in Stage 3).	Customer; Money (VIP); Menu (Critic)	1 / 2 / 2	Rifqi
MenuItem[3]	entity	Holds a dish's name, price in cents and preparation units, which cannot change once created. Rejects a blank name, a price of 0 or less, and fewer than 1 preparation unit. Does not know how it is displayed or ordered.	Menu, ComboMeal, Order	1	Danial
Menu	entity	Owns the list of dishes and hands out a read-only copy, so callers cannot change the menu. Stage 3 adds `withCombos()`, which appends the combo at the end so the standard dishes keep their positions. Does not decide which dish a customer orders.	Restaurant, MenuItem, ComboMeal	1, 3	Danial
ComboMeal	entity	A menu item made of two other menu items: price is 90% of their combined price, preparation units are their sum. Rejects a null	extends MenuItem; Money; built by Menu	3	Danial

[2]Rifqi: a first draft of my design-discussion text was generated by Claude and then reviewed and edited by me. See gai-paper-trail/rifqi-claude-log.md.

[3]Danial: Claude helped generate this table and the wording of each row.

Class	Pkg	Responsibility (and what it deliberately does not do)	Collaborates with	Stage	Owner
		component and a combo as a component. Does not change how Order or Chef treat a dish.			
Order	entity	Holds one dish, its locked price, cooking progress and status, and guards its own status changes (cook, serve, pay, cancel). Rejects a null dish and a price of 0 or less. Does not know its customer.	MenuItem, OrderStatus, Chef, Cashier, Waiter; held by Customer	1	Danial
Chef	entity	Does one preparation unit on an order it is given. Holds no state.	Order, GameController	1	Danial
Cashier	entity	Takes payment from at most one customer per call: the one who has waited longest, never one who became ready this turn. Returns the customer who paid, or null. Holds no state; leaves revenue and freeing the table to the controller.	Customer, Order, GameController	1	Danial
Staff (abstract)[4]	entity	Holds the one-action-per-turn rule once for every role: `startTurn()` resets it, `checkCanAct()` refuses a second action, `markActed()` uses it up. Each role names itself through abstract `getRole()`. Does not know what any action does.	InvalidActionException; extended by Waiter and Host	1	Kenz
Waiter	entity	Seats a queued customer, takes a seated customer's order, serves a READY dish, or waits, once per turn. Checks its rules before changing anything, so a refused action does not use up the turn. Does not set prices or decide when a dish became READY.	Staff, Restaurant, Customer, Order, Table	1	Kenz
Host	entity	Seats at most one queued customer per turn, on top of the Waiter's task, through `Restaurant.seat()`. Cannot take orders or serve.	Staff, Restaurant, Customer, Table	3	Kenz
Table	entity	Holds a table number and at most one customer. `seat()` refuses a taken table; `free()` empties it when its customer pays or leaves. Does not decide who sits where.	Customer; owned by Restaurant	1	Kenz

[4]Kenz: the content of my rows comes from report/design-discussion/kenz.md. See gai-paper-trail/kenz-genai-paper-trail.md for Claude's explanation and review (e.g. why Host should not extend Waiter).

Class	Pkg	Responsibility (and what it deliberately does not do)	Collaborates with	Stage	Owner
Restaurant	entity	Owns the tables, waiting queue, every customer who arrived, the menu and the staff. `seat()` is the only method that seats anyone and checks every rule first. Answers lookups (`freeTables()`, `customersToServe()`, `ordersToCook()`) and hands out read-only copies. Does not run the turn or keep score.	Table, Waiter, Host, Chef, Cashier, Menu, Customer, Order	1 (Host in 3)	Kenz
GameController[5]	control	Runs the game: owns the turn sequence (arrivals, Happy Hour offer and recovery, Host, Waiter, Chef after the Waiter, eating and payment, waiting and abandonment) and keeps the score (paid, revenue, average satisfaction). Catches InvalidActionException, shows the reason and asks again. Never prints or reads the console itself.	ConsoleUI, InputReader, GameConfig, HappyHour, Restaurant, Customer, PricingPolicy	1 (2: arrivals; 3: Host, Happy Hour)	Soon How
GameConfig	control	The fixed setting for one game: turns, tables, menu, arrival schedule, Happy Hour length and recovery, and the victory targets. Never changes once created; `base()` gives the reproducible setting. Refuses a schedule out of turn order or past the last turn.	Arrival, Menu, GameController	1 (2, 3)	Soon How
Arrival	control	One scheduled arrival: the turn and the customer type, given as a constructor reference (`IntFunction<Customer>`, e.g. `VIPCustomer::new`). A new customer type joins through the schedule; the turn sequence does not change.	Customer, GameConfig	2	Soon How
HappyHour	control	Timing only: whether Happy Hour has been used, whether it is on during a turn, and the turns left. Once per game, 2 turns from the turn it starts. The discount comes from PricingPolicy and the recovery is applied by GameController.	GameController, PricingPolicy	3	Soon How
ConsoleUI	boundary	Prints everything the player sees: restaurant state, each customer	Restaurant, Customer, Table, WaiterTask,	1 (2, 3)	Kenz

[5] Drafted by Claude (Anthropic) from the repository (feature/stage-3 merged with dev, 13 Nov 2026) and the members' report/design-discussion files; reviewed and edited by Soon How. Prompt and reply logged in gai-paper-trail/soonhow-claude-log.md.

Class	Pkg	Responsibility (and what it deliberately does not do)	Collaborates with	Stage	Owner
		with their type, choices, error reasons, running score, end-of-turn results, Host and Happy Hour prompts, final result. Holds no game rules and changes nothing.	Money		
InputReader	boundary	The only class that reads input. Asks again until it gets a whole number in range; stops with a clear error if a replay file runs out; echoes choices in piped runs. Does not decide whether a choice is allowed.	GameController	1	Kenz
CustomerState / OrderStatus / WaiterTask	enums	Fixed lists of states and tasks with display labels: CustomerState (WAITING, SEATED, ORDERED, EATING, READY_TO_PAY, PAID, LEFT); OrderStatus (stages from received to paid or cancelled); WaiterTask (SEAT, TAKE_ORDER, SERVE, WAIT). They do not change state themselves: Customer, Order and GameController do.	Customer; Order; ConsoleUI, GameController	1	Rifqi / Danial / Kenz
PricingPolicy	enums	Holds the percentage each pricing rule charges (NORMAL 100%, HAPPY_HOUR 80%) and works out an order's price, applied after the customer's own discount. Does not decide when Happy Hour is on.	Money, GameController, HappyHour	3	Danial
Money	util	Converts cents to a dollar string and works out a percentage of an amount, rounded to the nearest cent. Rejects a negative amount and a percentage outside 0 to 100. Holds no state or pricing rules.	ComboMeal, PricingPolicy, VIPCustomer, ConsoleUI	1	Danial
InvalidActionException	util	Checked exception for a choice that breaks a game rule; its message is the reason shown to the player.	thrown by Staff, Waiter, Host, Restaurant; caught by GameController	1	Kenz

3.2 Design principles shown in the code

Principle	Where (class or method)	What would go wrong without it
SRP	**PricingPolicy** (Danial): the Happy Hour rate sits in its own enum, not in `Money`, so Money only formats and rounds. **Restaurant.seat()** (Kenz): the seating rules live once; `Waiter.seat()` and `Host.seat()` only call it. **Customer.leave()** (Rifqi): only sets the state to LEFT; Order cancels, Table frees and the controller removes. **HappyHour** (Soon How): timing only; price and recovery are elsewhere.	Money would change for both display and promotions; the seating rules would be written twice and drift apart; Customer would need a reference to Restaurant and change whenever seating or payment rules changed.
OCP	**ComboMeal**[6] (Danial): adding combos needed no change to MenuItem, Order or Chef. ComboMeal extends MenuItem and supplies its own name, price and units. (Menu gained `withCombos()`.) **Customer subclasses + Arrival** (Rifqi, Soon How): Stage 2 added VIPCustomer and CriticCustomer and two schedule entries; Waiter, Chef, Cashier and Order were not changed. **Host** (Kenz): Stage 3 added a new Staff subclass; Staff, Waiter, Table and the seating rules were not changed.	Without these, every new dish, customer type or role would mean if/else or switch chains in Order, Chef, GameController and ConsoleUI, and editing working Stage 1 code.
LSP	**ComboMeal for MenuItem**: `ComboMealCheck` builds new `Order(saladPasta, saladPasta.getPriceCents())`, cooks it three times and gets NOT_READY then READY; Order has no code that mentions combos. **Any Customer subclass for Customer**: `VIPCriticCheck` holds all three types in Customer variables and gets losses 8/5/12 and Burger prices 1500/1350/1500 cents from the same calls. **Waiter and Host for Staff**: `HostCheck` shows each using its own one action in the same turn. This is also why Host does not extend Waiter (3.3).	Order, Chef and GameController would need `instanceof` checks for each kind of dish or customer. There is no `instanceof` in the project.
ISP	Not applicable. The project defines no interfaces of its own, and the brief warns against adding them only to show a principle.	–
DIP[7]	Partly. GameController's turn loop works only with the abstract `Customer` type, and new customers are created through Arrival's `IntFunction<Customer>`, so the controller never names a concrete customer class. Not claimed for the boundary: GameController depends on the concrete ConsoleUI and InputReader, which Main injects.	Without the abstract Customer type, adding the VIP and Critic would have meant editing the turn loop for each new type.

3.3 Trade-offs and one alternative design

- **Trade-off 1: one MenuItem class for all plain dishes**[8] (Danial). Salad, Burger and Pasta are MenuItem objects with different name, price and units, not subclasses. A new dish is one constructor call. The cost: a dish cannot have behaviour of its own.

[6] Danial: Claude suggested framing ComboMeal as an OCP example and explained the caveat that Menu was extended.

[7] Drafted by Claude (Anthropic) from the repository (feature/stage-3 merged with dev, 13 Nov 2026) and the members' report/design-discussion files; reviewed and edited by Soon How. Prompt and reply logged in gai-paper-trail/soonhow-claude-log.md.

- **Trade-off 2: a checked exception for rule breaks** (Kenz). `InvalidActionException` extends `Exception`, so the compiler forces every caller to handle a rule break, and a bad choice is always explained and retried instead of crashing the game. The cost: every action method carries `throws InvalidActionException`, and the controller needs a try/catch around each.
- **Trade-off 3: applyDiscount() has a default** (Rifqi). Regular and Critic pay full price, so the default in Customer avoids writing the same method twice. The cost: a new type that forgets to override it silently pays full price.
- **Alternative considered: Host extends Waiter** (Kenz). It would reuse `seat()`, but the Host would also inherit `takeOrder()` and `serve()`, so a Host could be used anywhere a Waiter is expected and told to serve food, which breaks LSP. Rejected; Host and Waiter are separate Staff subclasses and the shared seating rules live once in `Restaurant.seat()`.
- **Also considered:** one Customer class with a CustomerType enum and a switch in each method (Rifqi), and a subclass per dish (Danial). Both rejected: the first means editing several switches for each new type; the second gives empty classes with no behaviour of their own.

3.4 How the player's choices change the result

Targets: 4 paid, $45.00 revenue, average satisfaction 60. Numbers come from the recorded runs in testing/ unless marked.

Choice	Option A result	Option B result	Effect on targets
Use the Host and Waiter together, or let them wait[9]	stage3-victory: on turn 1 the Host seats Customer 1, the Waiter takes the order and the Chef cooks the Salad in the same turn. Served turn 2 at 92, paid $9.00 on turn 4.	stage3-defeat: Host and Waiter wait every turn. Customer 1 leaves at the end of turn 13 (13 × 8 = 104); the Critic leaves on turn 15. Nobody pays.	A: 5 paid, $75.24, 83 → Victory. B: 0 paid, $0.00, N/A → Defeat.
When to start Happy Hour	stage3-victory: started on turn 7, as the Critic arrives. The Critic's combo is billed $19.44 instead of $24.30, and at the start of turn 8 the Critic recovers from 88 to 100 (capped). The Critic pays at 76.	Started when nobody is waiting for a table or food: the 20% discount still applies to orders taken, but the +15 recovery helps nobody.	Costs $4.86 of revenue here, affordable because revenue ends $30.24 above $45.00. It protects the customer who loses satisfaction fastest, which matters for the 60 average.
Serve the Critic first or the VIP first	Critic first: it loses 12 a turn and leaves after 9 unserved turns (stage2-defeat: 9 × 12 = 108).	VIP first: a VIP loses only 5 a turn and needs 20 unserved turns to leave (100), longer than the game.	Serving the Critic first protects both the paid count and the average; the VIP can safely wait.
Cook the Burger or the combo first[10]	Burger first: Burger ready after 1 turn, combo after 4. Total waiting 5 turns.	Combo first: combo ready after 3 turns, Burger after 4. Total waiting 7 turns.	Burger first saves 2 customer-turns of waiting, which protects the average

[8]Danial: Claude explained that a trade-off and an alternative should each state their cost.

[9]Drafted by Claude (Anthropic) from the repository (feature/stage-3 merged with dev, 13 Nov 2026) and the members' report/design-discussion files; reviewed and edited by Soon How. Prompt and reply logged in gai-paper-trail/soonhow-claude-log.md.

[10]Danial: Claude suggested using specific examples to explain how a player's choice affects the result.

Choice	Option A result	Option B result	Effect on targets
			satisfaction.

4. GitHub evidence: the three stages

Each stage is a runnable tagged commit. Change tables below are taken from the README.[11]

Tag	Commit	Date	Runs on its own?
`stage-1`	`5609a16` github.com/armaldro/2003-Group3/tree/stage-1	11 Nov 2026	Yes: testing/stage1-victory-*, stage1-defeat-*, stage1-bad-input*
`stage-2`	`07aadcc` github.com/armaldro/2003-Group3/tree/stage-2	13 Nov 2026	Yes: testing/stage2-victory-*, stage2-defeat-*
`stage-3`	**6e00590 github.com/armaldro/2003-Group3/tree/stage-3**	13 Nov 2026	Yes: testing/stage3-victory-*, stage3-defeat-*

4.1 Stage 1 → Stage 2

Compare: https://github.com/armaldro/2003-Group3/compare/stage-1...stage-2

Each customer type's behaviour lives in its own class, so the new types needed no change to Waiter, Chef, Cashier or Order, and the only turn-sequence change is in `handleArrivals`.

Change	File	Why
New	entity/VIPCustomer.java	The VIP: loses 5 satisfaction per turn and gets 10% off (Rifqi)
New	entity/CriticCustomer.java	The Critic: loses 12 per turn, pays full price, always orders the dish that takes longest to cook (Rifqi)
New	control/Arrival.java	One scheduled arrival: the turn and which type of customer arrives (Soon How)
New	testing/VIPCriticCheck.java; testing/stage2-victory-*, stage2-defeat-*	Check the VIP and Critic; reproducible winning game (5 paid, $73.50, average 76) and losing game (Kenz, Rifqi)
Changed	control/GameConfig.java	The arrival schedule now says which type arrives each turn: VIP on turn 4, Critic on turn 10. Refuses a schedule out of turn order or past the last turn
Changed	control/GameController.java	`handleArrivals` asks the schedule to create each customer instead of hard-coding `new RegularCustomer(...)`; payment uses `Restaurant.getCashier()`
Changed	entity/Restaurant.java	Holds the Cashier through `getCashier()`
Changed	boundary/ConsoleUI.java, InputReader.java	Shows each customer's type through `getTypeName()` (no type checks), the running score, end-of-turn results and

[11] The README text these tables are taken from was drafted by Claude (Anthropic) and reviewed by Soon How (gai-paper-trail/soonhow-claude-log.md, entries 3, 6, 37 and later).

Change	File	Why
		"N/A" average; echoes piped input
Changed	entity/Customer.java, RegularCustomer.java, Staff.java, Table.java; enums/CustomerState.java, WaiterTask.java	`getTypeName()`; Javadoc fixes
Changed	testing/GameControllerCheck.java, testing/stage1-*-output.txt	Stage 2 checks; Stage 1 outputs regenerated with echoed input and customer type
Deleted	none	–

4.2 Stage 2 → Stage 3

Compare: https://github.com/armaldro/2003-Group3/compare/stage-2...stage-3

The Host is a new Staff subclass, the combo is a new MenuItem subclass, and the Happy Hour price is a separate PricingPolicy applied after the customer's own discount, so Waiter, Chef, Cashier and Order did not change. The turn gains two steps before the Waiter: the Happy Hour offer and recovery, then the Host.

Change	File	Why
New	entity/Host.java	Seats one waiting customer per turn, so the Waiter can take an order the same turn (Kenz)
New	entity/ComboMeal.java	Two dishes at 90% of their combined price, units summed (Danial)
New	enums/PricingPolicy.java	Normal price or Happy Hour price (80%), applied when an order is taken (Danial)
New	control/HappyHour.java	Happy Hour timing: used or not, on during a turn, turns left (Soon How)
New	testing/HostCheck, ComboMealCheck, MenuCombosCheck, PricingPolicyCheck, CustomerCheck, SatisfactionRecoveryCheck, HappyHourCheck	Checks for each new class and rule
New	testing/stage3-victory-*, stage3-defeat-*	Reproducible winning game (5 paid, $75.24, average 83) and losing game (Soon How)
Changed	control/GameConfig.java	Holds the menu, Happy Hour length (2) and recovery (15). The Critic moves to turn 7: as customer 3 it orders the combo where a Regular would order Pasta, so its twist still shows
Changed	control/GameController.java	Happy Hour step and Host step before the Waiter; Happy Hour price applied when an order is taken; menu from the setting
Changed	entity/Menu.java	`withCombos()`: the standard dishes with the combo appended (Danial)
Changed	entity/Customer.java	`recoverSatisfaction(points)` and `MAX_SATISFACTION` (Rifqi)
Changed	entity/Restaurant.java, boundary/ConsoleUI.java	Holds the Host (`getHost()`); shows Host choices, the Happy Hour offer and status (Kenz)
Changed	testing/GameControllerCheck.java	Stage 1 and 2 checks decline Happy Hour and let the Host wait; new Stage 3 checks
Deleted	none	–

5. Functional evidence and testing

Every run below was replayed on tag stage-3 (commit 6e00590) with java -cp out Main < testing/<input file>, and the output matched the committed output file exactly. testing/stage3-bad-input.txt was recorded after the tag on the same game code; only the test files were added. The Stage 1 and Stage 2 replays (testing/stage1-* and testing/stage2-*) match the game at their own tags, stage-1 and stage-2.

5.1 Reproducible runs

Run	Input file	Output file	Result	Paid	Revenue	Avg satisfaction
Victory	`testing/stage3-victory-input.txt`	testing/stage3-victory-output.txt	Victory	5	$75.24	83
Defeat	`testing/stage3-defeat-input.txt`	testing/stage3-defeat-output.txt	Defeat	0	$0.00	N/A (Customer 1 and the Critic left)
Bad input	testing/stage3-bad-input.txt	testing/stage3-bad-input-output.txt	5 invalid entries on turn 1 rejected with a reason (two non-numbers, two numbers out of range, one Waiter task not open that turn); none uses up a turn, and turns 2 to 18 then match the defeat run	–	–	–

Replay with `java -cp out Main < testing/<input file>` (Git Bash, macOS, Linux) or `Get-Content testing\<input file> | java -cp out Main` (PowerShell).

5.2 Feature evidence

Feature	Evidence (console extract)	Caption

Start-of-turn display	`--- Turn 7 of 18 ---`
	`Ding! Customer 3 (Critic) has arrived.`
	`Revenue: $9.00 | Average satisfaction of paid customers: 92`
	`Tables:`
	`  Table 1: free`
	`  Table 2: Customer 2 (VIP)`
	`Waiting queue: Customer 3 (Critic)`
		stage3-victory-output.txt, turn 7: arrival, running score, tables, queue and each customer with type.

Dish READY → SERVED → PAID	`Customer 3 (Critic): ORDERED, satisfaction 76, Salad + Pasta (Ready)`
	`Customer 3 (Critic): EATING, satisfaction 76, Salad + Pasta (Served)`
	`Customer 3 (Critic): READY_TO_PAY, ... (Served)`
	`Customer 3 (Critic) paid $19.44.`
		stage3-victory-output.txt, turns 10 to 12: served the turn after it became READY, eats one turn, then pays.

VIP discount and Critic behaviour	`Customer 2 (VIP) paid $13.50.` `Customer 3 (Critic): ORDERED, ..., Salad + Pasta`	VIP pays $13.50 for a $15.00 Burger (10% off). The Critic, as customer 3, orders the slowest dish (the combo) where a Regular would order Pasta.

Host seats an extra customer	`Host tasks:`
	`  0. Wait`
	`  1. Seat Customer 3 (Critic): WAITING, satisfaction 100`
	`Host action: 1`
	`Waiter tasks:`
	`  2. Take an order`
	`Waiter action: 2`
		stage3-victory-output.txt, turn 7: the Host seats the Critic, so the Waiter takes the order in the same turn.

ComboMeal price (Salad + Pasta = $24.30)	`Customer 4 (Regular) paid $24.30.`	Regular customer 4 orders the combo outside Happy Hour: 90% of $9.00 + $18.00 = $24.30, cooked over 3 units.

Happy Hour: 20% off, +15, turns left	`Happy Hour action: 1`
	`Happy Hour: ON, 2 turn(s) left`
	`...`
	`Customer 3 (Critic): ORDERED, satisfaction 88, ...`
	`Happy Hour: ON, 1 turn(s) left`
	`Customer 3 (Critic): ORDERED, satisfaction 100, ...`
	`...`
	`Customer 3 (Critic) paid $19.44.`
		Started on turn 7; on turn 8 the Critic recovers 88 → 100 (capped); the combo ordered during Happy Hour bills $24.30 × 80% = $19.44.

Feature	Evidence (console extract)	Caption

Customer leaves at 0 satisfaction	`Customer 1 (Regular) ran out of patience and left.`
	`End of turn 13: served 0, unhappy departures 1, ...`
	`Customer 1 (Regular): LEFT, satisfaction 0`
		stage3-defeat-output.txt, turn 13: a Regular never served leaves after 13 turns (13 × 8 = 104).

5.3 Rule checks

The check classes in testing/ are plain `main` programs that print OK or FAIL per check.

Check class	What it verifies	Result
GameControllerCheck	The Stage 1 turn sequence and scoring, the Stage 2 arrival schedule with the VIP and Critic in play, the Stage 3 Host, Happy Hour, recovery and the Critic on turn 7, and full winning and losing games	OK
HappyHourCheck	Happy Hour unused at first, on for 2 turns from the turn it starts, counting down, then off; usable once per game only	OK
CustomerCheck	Customer starting values, satisfaction loss, state changes, full price and the food rule	OK
VIPCriticCheck	VIP and Critic loss (5, 12), prices (1350, 1500 cents for a Burger), dish choice and type names, through Customer variables	OK
SatisfactionRecoveryCheck	`recoverSatisfaction()`: +15 per Happy Hour turn while waiting, never above 100	OK
WaiterCheck	Table, Staff, Waiter, Restaurant and InputReader: free and occupied tables, seating, taking an order and serving, one action per turn	OK
HostCheck	Host seats at most one queued customer per turn, on top of the Waiter, through the same seating rules	OK
MenuItemCheck	MenuItem rejects a blank name, a price of 0 or less and fewer than 1 unit	OK
MenuCheck	The standard menu (Salad, Burger, Pasta) and its read-only copy	OK
MenuCombosCheck	`withCombos()` has 4 items: Salad 900/1, Burger 1500/1, Pasta 1800/2, Salad + Pasta 2430/3	OK
ComboMealCheck	Salad + Pasta is 2430 cents and 3 units; Burger + Salad 2160 and 2; rejects null and combo components; cooks through NOT_READY to READY in an Order	OK
PricingPolicyCheck	NORMAL keeps the price; HAPPY_HOUR 1500 → 1200, 1800 → 1440; VIP then Happy Hour on 1500 → 1080; combo 2430 → 1944	OK
OrderStatusCheck	Each order status and its display label	OK
OrderCheck	A 1-unit dish goes straight to READY; each illegal move (serve or pay too early, cook a READY or CANCELLED order, cancel a SERVED order) throws IllegalStateException; cancelling is allowed before serving	OK (9 checks)
ChefCheck	Pasta NOT_READY after 1 call and READY after 2; Burger READY after 1; cooking a READY or CANCELLED order is refused	OK
CashierCheck	Nobody pays from an empty list or while WAITING; a customer ready this turn does not pay; the oldest ready customer pays; an ineligible first	OK

Check class	What it verifies	Result
	customer does not block an eligible second	
MoneyCheck	`format()` for 0, 5, 50, 100, 1080 and negative amounts; `percentOf()` rounding (e.g. 1999 × 15% = 300, 5 × 50% = 3), chained VIP then Happy Hour discounts, and rejection of a negative amount or a percentage outside 0 to 100	OK (14 checks)

6. Build, run and Javadoc

Requires JDK 17 or later. In Git Bash, macOS or Linux:

```
# compile

javac -d out $(find src/main/java -name "*.java")



# run

java -cp out Main



# replay a recorded run

java -cp out Main < testing/stage3-victory-input.txt



# generate Javadoc

javadoc -d docs/javadoc -sourcepath src/main/java -subpackages
entity:control:boundary:enums:util
```

In Windows PowerShell, compile and replay differ; run and Javadoc are the same:

```
javac -d out (Get-ChildItem src\main\java -Recurse -Filter *.java).FullName

Get-Content testing\stage3-victory-input.txt | java -cp out Main
```

Javadoc is in docs/javadoc/index.html. Every public class and method carries Javadoc.

7. Individual reflections

7.1 Muhamad Rifqi bin Zulkiflie

My part was the customers: CustomerState, the Customer class and RegularCustomer for Stage 1, then the VIP and Critic for Stage 2, the Happy Hour recovery for Stage 3, plus the class diagram. Before this I had only done Python in CT2001, so Java and GitHub were both new to me. My work laptop does not allow installing Git, so I ended up doing everything in GitHub Codespaces and picked up branches and pull requests as I went. The part that made OOP click for me was the abstract class. I wrote loseSatisfaction() once in Customer, and each type just says how much it loses, so when the VIP and Critic came in Stage 2, nobody had to touch the game loop. One mistake I made was copying a method and forgetting to change the field name, so the ready-to-pay turn was saved in the wrong place. Java did not complain, so now I always check with real numbers. If I did it again, I would start the class diagram much earlier and write the checks together with the code.

7.2 Muhammad Danial Haziq bin Burhan

I did the entity classes Chef, MenuItem, Menu, Order and Cashier, the Money utility and the OrderStatus enumeration in Stage 1, and the ComboMeal entity class, PricingPolicy and the Menu.withCombos() method for Stage 3, as well as design decisions. I have a clearer understanding of OCP through the ComboMeal class extending MenuItem. Without this approach, I would have to code if/else statements in every class that involves MenuItem just to accommodate a ComboMeal. This showed me why extending a class is important, especially when accommodating more updates in the future when code becomes more complex. A mistake I made was in MenuItemCheck, where it printed FAIL for every correct rejection, because it treated a thrown exception as a failure without knowing what I expected. Reading it from the terminal made no sense because the behaviour was as expected but it printed FAIL. Afterwards, I changed it so that if the test caught the error properly, it would print OK, and FAIL if it couldn’t. I also printed the expected and actual output so that it would be clearer to see what went wrong when I run the check. If I started the project again, I would create a helper class for testing because I noticed much of the testing used repeated code.

7.3 Muhammad Kenz s/o Jafar

My part was the tables, staff and console: Table, Staff, Waiter, Restaurant, ConsoleUI and InputReader in Stage 1, showing each customer's type in Stage 2, and the Host in Stage 3. I think my classes ended up small and well tested. WaiterCheck and HostCheck both pass, and the Stage 2 replays I recorded gave exactly the result we planned: 5 paid, $73.50 and an average satisfaction of 76.

What I could have done better is my time. I started late, and to keep up I asked Claude to write a lot of my code, including Restaurant, Waiter and Host. I am sad about that. I can explain the code, but I did not work through most of it myself, and that is how I learn best. It also left me with a question I can't fully answer: AI is incredibly useful for writing code, but if it writes the code and I only study it, at what point does that become cheating? I declared all of it in my paper trail, but the question still bothers me.

Using GitHub as a team was new to me. Branches and pull requests let four of us build one project and merge it together, which worked well. The downside was waiting. I could not record a winning

game until the Cashier and the turn loop were merged, so my work was often blocked on someone else's.

On OOP, my own classes taught me the most. Staff holds the one-action-per-turn rule once, so Waiter and Host just inherit it, and Restaurant.seat() is the only place anyone is seated, so the seating rules are never repeated. Polymorphism clicked when ConsoleUI showed VIP and Critic customers through getTypeName() with no instanceof check. I also learned why Host should not extend Waiter: it would inherit takeOrder() and serve(), which breaks the Liskov Substitution Principle.

7.4 Teo Soon How

I designed the turn sequence and scoring, wrote the Stage 3 phases (HappyHour, the Happy Hour step and the Host step), and handled integration: merging pull requests, the stage tags and the README. My hardest problem was timing: whether something happens this turn or next. My first Happy Hour version was offered before the restaurant state was shown, and gave the +15 after the waiting loss, although Rifqi's Javadoc said it applies at turn start. I moved both, and the state is now shown again after the recovery, so the numbers on screen match. The OO lesson for me was one method per phase. My first GameController did everything in start(); after a review I split it, and Stage 3 then needed only two new methods, happyHourPhase() and hostAction(), two calls in playTurn() and one pricing line in takeOrder(), with the Stage 1 phases untouched. Next time I would agree the timing rules with the team before coding, and merge into dev earlier in smaller pieces.

8. GAI acknowledgement, appendices and submission checklist

8.1 Use of generative AI

Each member lists their own use. The paper trail in gai-paper-trail/ holds the prompts and outputs, and any idea or text from GAI is footnoted where it appears in this report.

Member	Tool	What it was used for	Paper trail file
Rifqi	Claude (Anthropic), via Claude Code	Explaining concepts and reviewing my code; generating chooseItem, VIPCustomer, CriticCustomer, recoverSatisfaction, getTypeName, my checks and the class diagram; drafting my report text (footnoted).	gai-paper-trail/rifqi-claude-log.md
Danial	Claude Code (Claude Sonnet 5.5)	Skeletons with TODOs of OrderStatus, Order, Cashier, ComboMeal, PricingPolicy and Menu.withCombos(); check files MenuItemCheck, MenuCheck, OrderCheck, ChefCheck, CashierCheck, ComboMealCheck, PricingPolicyCheck, MenuCombosCheck; explanation of SOLID and review of my classes. All method logic written by me.	gai-paper-trail/danial-claude-log.md
Kenz	Claude (claude.ai) and Claude Code (Claude Opus 5.5)	Boilerplate for Table, WaiterTask, InvalidActionException; skeletons with TODOs for Staff, Waiter, Restaurant, InputReader; code for the Restaurant and Waiter TODOs, InputReader.readChoice(), Host, getHost(), getCashier(), the ConsoleUI display methods, Javadoc fixes, WaiterCheck, HostCheck and the Stage 2 replay files; explanation and review. Staff TODOs written by me with guidance.	gai-paper-trail/kenz-genai-paper-trail.md
Soon How	Claude (Anthropic), via Claude Code and claude.ai	README text; GameConfig; the GameController turn sequence and scoring (from my plan and design decisions); Arrival and the Stage 2 arrivals; ConsoleUI display methods; GameControllerCheck and the replay files; design ideas and branch reviews; compiling and drafting parts of this report (footnoted).	gai-paper-trail/soonhow-claude-log.md

8.2 Appendices

- A. Signed Declaration of Original Work (in forms/)
- B. Declaration on Use of GAI Assistance, one per member (in forms/)
- C. Full console output of the victory and defeat runs: testing/stage3-victory-output.txt and testing/stage3-defeat-output.txt

8.3 Submission checklist

Zip name: **COMP1_Grp3.zip**

☐ Final class diagram matches the stage-3 code and is annotated (HappyHour still to add)

☑ Design discussion has the abstractions table, SOLID only where real, trade-offs, one alternative, player choices

☑ Tags stage-1, stage-2, stage-3 on runnable commits; README lists changed, new and deleted files with reasons and compare links

☑ Victory and defeat runs reproducible from testing/

☑ Javadoc generated for every class

☑ Four reflections in

☑ Declaration of Original Work signed; four GAI declarations; paper trail included

☐ Report exported to PDF and placed in report/

☐ Zip uploaded to NTULearn by Fri 13 Nov, 11:59 PM